Wizardry's Party of Six Returns, With Friends and AI Companions Who Remember You and Can Die

crypt fills every seat in the party with any mix of human players and companions, and with a model connected gives Zork and Wizardry fans a dungeon that never runs out

Built to work through Chip Huyen's AI-engineering curriculum end to end, with a game people enjoy as the proof

Press Release

SAN FRANCISCO: Today the crypt project released crypt, a text-adventure dungeon crawler for players who grew up on Zork, Wizardry, and the old MUDs. A party descends together, with up to six seats, filled in any mix of friends and AI companions. Each companion has its own personality and voice, remembers what happened in earlier sessions, and can die. A deterministic engine enforces every rule for humans and companions alike, the way a board game does. Reloading a save never brings a dead companion back. Where the dungeon offers resurrection, a companion pays the same price a human does. Under the party, with a model connected, a language model writes a new dungeon for every descent, so the dungeon never runs out (see FAQ 11).

Problem

The dungeon crawls that defined a genre ran out of content. Zork had to fit a 32K microcomputer, so its authors cut it from 191 rooms to 110 and its vocabulary from 911 words to 617 [1]. Wizardry shipped ten fixed levels under one castle [2]. Once a player solves them, the mystery is gone. The MUDs answered with crowd authorship: LPMud wizards and LambdaMOO builders added rooms, monsters, and objects from inside the running game [3, 4]. Crowd authorship needs a crowd, and a world grows only while its builders keep logging in. The party had its own limit. Wizardry gave one player six characters to run alone [2]. A MUD offered other people, but a party still needed five of them free on the same evening. Games run entirely by a language model promise endless content and then lose track of it. The top models in RPGBench's evaluation write engaging stories while keeping the mechanics inconsistent [5]. AI Dungeon's own help page explains that older story text is trimmed once the context fills [6].

Solution

A player opens a party and fills its six seats in any mix. A player alone starts with three companions and can add up to five. Two friends and three companions make a full party, and so do six friends. A companion is a character played by a language model, with a class, a personality, and a voice of its own. It remembers the last descent: who took the cursed ring, who ran from the ogre, who carried whom out of the pit. It acts through the same moves a human player has, and the engine resolves its sword swing or its spell with the same rules and the same dice. When a companion dies, no reload undoes it; where the dungeon offers resurrection, a companion pays the same price a human does. Under the party, with a model connected, every descent after the first bundled one is a freshly generated dungeon. Before a dungeon opens, the engine walks it to prove that every locked door has a key and every goal can be reached. Around the text, each companion speaks aloud in its own voice. Richer presentation follows as each layer proves it can keep up without making the player wait: a score that shifts with the scene, then panels that redraw around it (see FAQ 21).

Customer Quote

> *"In Wizardry I ran all six characters myself. Now my son plays the cleric, I play the fighter, and the other four are companions. Three sessions ago I sold an amulet our thief, Wren, had found, to pay for my own resurrection. Last night, splitting the loot from the lich's vault, she brought it up and would not let it go. An hour later she held the bridge so we could run, and the ogre killed her. The temple wanted more gold than the party had, and reloading does nothing. She is gone. We have a new thief now, and my son still won't talk to him."*
>
> **Dave Whitman**, lifelong text-adventure player

Getting Started

A player installs crypt, runs it, and is in a bundled dungeon within a minute, with three companions already in the party. Inviting a friend hands that friend a seat. From inside Claude Code, the game connects through a built-in bridge and Claude becomes the dungeon master directly. From a terminal, the game runs its own client and talks to whatever model the player has available, down to a fully offline mode that still plays with built-in narration. A player joins the hosted service with an invite link. There is no account at launch; an invite link is the identity. There is no world to download. A dungeon is one command away (see FAQ 7).

Spokesperson Quote

> "I built crypt to learn how to engineer an AI-enhanced product: where to put the model, how to measure whether it is any good, and how to tune it. A companion is the hardest exercise I could find: an agent with a memory and a personality that has to play fair inside rules it cannot bend. The engine knows which moves are legal and what happened, so I can score every companion's rule-following against it, and every memory claim about something the engine recorded. How well a companion chooses, and whether it stays in character, I score with judges. The game is the part people see; the learning is why it exists."
>
> **Jim Freeman**, crypt project

Call to Action

crypt is free and open source. A player can join a party on the hosted crypt service with an invite link or run the game on their own machine. To play it, build it, or read how the engine and the companions divide the work, visit the repository at github.com/punt-labs/crypt.

Frequently Asked Questions

External FAQs

Q1: What is crypt and who is it for?

crypt is a text-adventure dungeon crawler for players who loved Zork, Wizardry, and the MUDs. Its center is the party: six seats, filled by friends, by AI companions, or by both. Companions remember earlier sessions, keep their own personalities, speak in their own voices, and can die, with no reload to undo it (see FAQ 4). The classic mechanics run in a deterministic engine: grid dungeons, turn-based combat, character classes, inventory, and leveling. With a model connected, a language model writes a new dungeon for each descent, so the party never runs out of places to go. In rules-only play the game serves hand-authored scenarios from a bundled set (see Feature 1).

Q2: How is this different from AI Dungeon, or from other games that pair an engine with a language model?

crypt plays at Wizardry's tempo: one keypress per decision, in a corridor or in combat, with the party's state always on screen. Its terminal client has no page to freeze, and vox plays each voice in a separate process. The engine answers the moment a key is pressed, and narration arrives behind it. The closest commercial neighbor, TableForge, plays as a browser chat surface. In one free-tier session, a turn was a mouse path: open Quick Actions, pick an action chip or type, press Send, and wait. By default voice played only when the player clicked a play button under each message, party state sat behind tabs, and nothing observed was keyboard-reachable. The wait appeared to have two layers. One was model and voice synthesis time, the layer crypt's predictive cache targets. The other was the browser laboring under the page itself, and the tab froze once and had to be reloaded. The session was slow enough to doubt it holds attention for hours [7].

AI Dungeon is collaborative storytelling with no rules engine. Its multiplayer mode sets no limit on players in one story [8], and the story lives in the model's context, which trims older text as it fills [6]. In crypt, hit points, inventory, and death live in the engine and do not drift as a session grows long.

TableForge's makers say it gives "final authority over dice, damage, conditions, and resources to code." An AI dungeon master narrates on top, a party seats up to six human players, and one to four AI companions join depending on the plan [9, 10, 11]. Friends & Fables, Hidden Door, Auferet, and Questie each cover part of the same ground [12, 13, 14, 15]. On the pages read in November 2026, none of six reviewed products stated five things a crypt player gets. A crypt player can read the source and play offline with no model at all. A player inside Claude Code brings their own model, and it runs the game through the engine's tools. A companion who says she was cheated out of an amulet three sessions ago is checked against what the engine recorded. A companion who fell to the ogre stays dead through the game's own save and load, and on the hosted service. A score and panels follow the scene as each passes its gate. Hidden Door comes nearest on two of these. A world creator can set characters to permadeath. Its Creator MCP lets Claude author worlds, and Hidden Door's own story engine runs play [13]. At least eight open-source games also pair a rules engine with a language model, among them kliuchnikovv/dnd, Mnehmos/rpg-mcp, open-dungeon, and a fork of Lebbitheplow/open-game-master [16, 17, 18, 19] (see FAQ 17).

Q3: Can I play with friends, or do I have to play alone?

Both, in one party. A player alone starts with three companions and can add up to five. A player and two friends leave three seats for companions. Six friends fill all six and play with no companions at all. A friend can join between descents and take a companion's seat, and the companion steps aside with its memory intact (see Feature 3).

Q4: What happens when a companion dies?

It stays dead, the same as a human player's character. The engine records the death the moment it happens, and reloading a save cannot undo it. Where the dungeon offers resurrection, at a temple or through a spell, it follows the same rules and costs the same for humans and companions. A fallen companion's seat opens for a new companion, a new character, or a friend. The game's own save and load never undo a death, and the hosted service enforces it. A local player with the source can copy a save directory, as Wizardry players copied disks; the game does not try to stop that (see Feature 2).

Q5: Do I need an API key or an internet connection to play?

No. crypt runs in tiers. A large model accepts the widest range of free-text commands, writes the longest and most varied descriptions, and gives companions the most detailed recall and dialogue. A small local model still generates, narrates, and plays companions, with shorter descriptions and a narrower range of commands it understands. With no model at all, the game falls back to built-in rules and templates, companions fight by scripted tactics, and the game stays fully playable offline.

Q6: Will a companion or the narrator cheat, forget my inventory, or break the rules?

No. A model never decides a game outcome. Every rule, every die roll, every point of damage, and every saved game is handled by the engine in code. A companion chooses what to try, the same as a human player, and the engine decides what happens. The narrator describes the result, but it cannot change your hit points, revive a dead companion, or hand out a sword the dungeon does not hold (see FAQ 12).

Q7: How do I get started?

Three ways in. On the hosted service, a player opens an invite link and is in a party with three companions. There is no account at launch; the invite link is the identity. On a terminal, a player installs crypt from the repository, runs it, and plays against whatever model is configured. That can be a model provider key or a small local model. With no model at all, the game plays fully offline with built-in narration and scripted companion tactics. Inside Claude Code, a player adds the crypt bridge and Claude becomes the dungeon master, running the game through the engine's tools (see Feature 8). In every mode a bundled scenario opens within a minute of starting. A generated dungeon opens after the engine's solvability walk passes, and the validation run measures how long that takes.

Q8: How much does it cost?

Nothing. crypt is free and open source. Run locally, it uses the player's own model provider key, a small local model, or no model at all. Any model cost is the player's own, and it can be zero. The hosted service is free and invite-only, and it runs under a monthly spending ceiling. At the ceiling, the hosted game drops to the small-model tier, then to rules-only play, and new invites pause until the next month. In rules-only play no model generates dungeons, so the game serves hand-authored scenarios from the bundled set, which is finite (see Feature 13).

Q9: What happens to my data?

What a player types and what each companion remembers are game state, and the server stores them with the rest of the game. On the hosted service, that text goes to the model provider the operator configures, because the model reads it to interpret commands and play companions. In local mode it goes wherever the player's own model runs, and with a local model or no model it never leaves the player's machine. There are no accounts at launch, so the hosted service collects no email address; an invite link is the identity. The engine bounds what prompt injection can do to outcomes: an injected instruction cannot reposition a companion, change a roll, or alter inventory. What it can change is what a companion or the narrator says. On the hosted service, generated text passes an output check before it reaches other players, and the check is one of the Chapter 10 guardrails exercises [20]. The hosted service is not designed for children, and anything typed into it reaches a model. A family that wants everything kept on its own machine can run crypt locally.

Internal FAQs

Purpose & Learning Objectives

Q10: Why build this? What is the real objective?

crypt is a build-to-learn project, roughly four parts learning to one part game. The curriculum is Chip Huyen's *AI Engineering* [20]. Chapters 3 through 10 each get a concrete exercise in a running application instead of a toy notebook; Chapters 1 and 2 are background reading (see FAQ 15). Her central discipline is evaluation-driven development [20]. In crypt the deterministic engine knows which moves are legal and what happened, so rule-following and memory have a recorded answer to score against. The dungeon crawl is the vehicle, and I grew up playing the games it draws on; the understanding is the product. The finish line is a public, playable game running in production on AWS, with per-call latency, tokens, cost, eval score, and error rate recorded for every model call. Production work is part of the curriculum: Huyen's chapters on inference optimization and on monitoring and the feedback flywheel need a live system to practice on (see Feature 12).

Q11: Why is the party the center of the product?

Three reasons: the player, the curriculum, and the evidence. Wizardry's party of six is the part of the genre I miss most, and playing with friends or with model-played companions is the hook I want for myself. In Huyen's curriculum, a companion is the deepest agents exercise in the project: it plans, calls tools, keeps a memory, and fails in ways that need guarding against. Generative Agents is the peer-reviewed anchor for that design. Park et al. gave 25 language-model agents memory, reflection, and planning, and the agents behaved believably over days of simulated life [21]. A candidate dataset exists. FIREBALL holds about 25,000 sessions of D&D 5e play recorded through a Discord bot, with structured game state [22]. Whether it transfers to how players act in a crypt party is untested, and its license terms for training use are checked before any fine-tuning run.

Build order follows from the party. Companions come before panels, because a beautifully presented room with nobody worth adventuring beside is the failure to avoid. Generated scenarios supply the places the party goes (see Feature 9). Presentation comes last and deepens only as far as its gates allow (see FAQ 21).

Q12: Where is the boundary between classical code and the language model?

The engine owns anything that must be exact and repeatable: state transitions, combat resolution, inventory limits, movement, death, and save and load. The models own anything that benefits from creativity and language: generating a scenario, describing a room, interpreting free-text commands, and playing a companion. A companion acts only by calling the same engine tools a human player's commands reach. It can choose to attack, cast, flee, or argue over the loot, and the engine resolves the attempt by the rules. A companion's memory and personality live outside the engine. They decide what the companion tries; the engine decides what happens. Generated presentation sits on the same side as narration: a panel, a score, or a voice displays engine state and never changes it.

The evidence for keeping state in code is consistent. RPGBench finds that the top models in its evaluation write engaging stories as game engines while leaving the mechanics inconsistent [5]. Song et al. show that giving a game master function calls for state improves both narration and state consistency [23]. Infobip reports taking dice and turn order away from the model after

its pure-language-model D&D engine fell below 65% accuracy as context grew. That figure is a vendor self-report with no published method [24].

Other systems let a model enforce the rules. CriticaLLM pairs a game-master model with a judge model that validates player input [25]. Bicking's Intra separates some state from the model but leaves outcome resolution to the model [26]. A study of hybrid narrative managers at ICTAI 2026 compares several designs and finds that none dominates on every measure [27]. crypt keeps rules in code because the engine's answer can be checked. The same state and the same action give the same result, and a test can assert it. A judge model's ruling has no such check, so a wrong ruling looks the same as a right one. The engine owning state is the reason a companion's death stays final and its memory of an amulet matches what the engine recorded. The exact placement of the line for each new capability, such as how much a companion may propose before the engine rejects it, is tested by building.

Q13: How will the model's performance be evaluated?

With all three of Huyen's evaluation approaches [20], anchored where possible to what the engine records. Exact evaluation covers three things. The interpreter is scored against a hand-labeled set of player inputs and their intended actions. Agreement with the rule-based parser is reported only on inputs the rule-based parser handles. Balance is measured by running a fixed set of automated sessions, 200 per scenario, against survival and progression targets. Companion rule-following is exact too: every tool call a companion makes is checked against what the engine allowed. Memory is exact for claims about recorded events, after one extraction step. A judge model pulls factual claims out of companion dialogue, such as who sold the amulet or who fled the ogre. Each extracted claim is then checked exactly against the engine's event log. The exact score inherits the extractor's recall, which is reported alongside it.

AI-as-a-judge covers what the engine cannot score: whether a companion's choice was a good one, whether its personality holds across sessions, and narration quality. Tester personas do the judging. Their scores are treated cautiously, because Huyen shows that a judge's scores drift over time [20]. For generated panels, a judge also scores multi-turn consistency: whether the screen still reads as the same game on turn two hundred as on turn one. EvoGenUI-Bench measures that property for generated interfaces [28]. Comparative evaluation ranks one model or prompt against another, which Huyen finds more reliable than absolute scores [20]. These feed a private leaderboard that ranks each model tier by cryptd's own criteria, so model and prompt choices are measured before they ship (see Feature 10).

Q14: What AI-engineering skills is this meant to build?

Seven, in order of depth. Integrating a model into a working application behind a clean boundary. Designing fallback tiers so the game keeps every rule and stays playable, with plainer narration, when the model is weak or absent. Building evaluation that scores model output against ground truth. Tuning prompts and swapping models to raise those scores. Building agents with memory and personality that act only through tools. Serving several models per turn within a latency and cost budget, in production, with monitoring. Fine-tuning a small model for the game's narrow narration, parsing, and companion tasks. Each builds on the one before, and the game gives each a concrete place to practice.

Q15: How does the project map to Chip Huyen's curriculum?

Chapters 3 through 10 of *AI Engineering* [20] each get a concrete exercise in crypt, and the milestones are sequenced to match. Chapters 1 and 2, an introduction to building with foundation models and an account of how those models work, are background reading with no exercise of their own. Evaluation methodology and evaluating AI systems (Ch 3 and 4) become the evaluation suite and the private leaderboard that rank the model tiers. Prompt engineering (Ch 5) becomes the interpreter and narrator system prompts, plus the persona and defensive prompts that keep a companion in character and inside the rules, scored for sensitivity when they change. Chapter 6 covers retrieval and agents, and the companions exercise both. Companion memory is retrieval: before a companion acts, it retrieves from the engine's recorded events and from its own earlier reflections. Its agent side is planning and action only through the engine's tools [21]. The game-as-dungeon-master and the tester personas are agents over the same tools. Fine-tuning (Ch 7) is an experiment, run whatever its outcome: train one small model on the harvested corpus and compare it to prompting on the leaderboard. A tuned model ships only if it wins that comparison. Dataset engineering (Ch 8) becomes a corpus of intent-to-action, event-to-narration, and companion-decision pairs, harvested from real and automated sessions. FIREBALL, D&D 5e play recorded through a Discord bot, is a candidate addition [22]; its transfer to crypt is untested, and its license terms for training use are checked before any fine-tuning run. Inference optimization (Ch 9) becomes the latency budget for a turn with several companions and the predictive cache that prepares the next screen while the player reads. Architecture and user feedback (Ch 10) becomes production on AWS. Every model call records latency, tokens, cost, eval score, and errors, and player comments and persona reports feed the next iteration.

Audience & Demand

Q16: Who is the audience, and is demand a concern?

The audience is players nostalgic for Zork, Wizardry, and MUDs who miss adventuring in a party and want a dungeon that does not run out. Demand is a minor concern here by design. This is a learning project, roughly four parts learning to one part game. Success is a public, playable game that records per-call latency, tokens, cost, eval score, and error rate for every model call. Demand still has a floor, because Huyen's chapter on architecture and user feedback (Ch 10) needs players. Its monitoring and feedback exercises need at least 10 returning players over a month. A returning player is one saved party that plays on two or more distinct days in a month; the save identity, not the invite link, is the unit. Below that number, those chapters run on persona play-testers, and the results are labeled as persona results (see FAQ 25). Any wider demand claim is a hypothesis with no primary research behind it yet. `[CITATION NEEDED]`

Competition

Q17: Who else builds games like this, and where does crypt stand?

Pairing a rules engine with a language model is an established pattern, commercially and in open source.

Six commercial products were reviewed from their own pages in November 2026. TableForge is the closest neighbor. Its makers say it gives "final authority over dice, damage, conditions, and resources to code" and uses AI "for interpretation and narration" [11]. A host's plan sets the party: up to six human players and one to four "travelling NPC companions that fight beside

you," with "persistent cross-session memory" claimed for the campaign [9, 10]. Friends & Fables seats up to six players around an AI game master that "narrates, adjudicates rules, and reacts to your world." Its home page names no AI companions [12]. Hidden Door runs a story engine that checks every beat "against canon," offers world creators a "None / Normal / Permadeath" setting, and has a Creator MCP for building worlds from Claude [13]. Auferet has a dice roller and stat tracking, but "the AI runs the world, the NPCs, and the outcomes." Its page warns that "AI narration can still apply a rule incorrectly" [14]. Questie's Party Mode seats one human with up to four voiced AI companions and has no rules engine beyond a d20 roll [15]. AI Dungeon's multiplayer has no rules engine and no set player limit [8].

One free-tier TableForge campaign was also played hands-on, in Safari on macOS [7]. The engine owns the numbers, as the vendor says: a 5e character sheet, a grid map, and a quest log sit in tabs beside the chat. A turn is a run of clicks: Quick Actions, an action chip or typed text, Send, then a play button under each message to hear it. Group voice autoplay exists as a setting and defaults off. The session felt sluggish, and the browser tab froze once and needed a reload. Companions were not seated on the free tier played; the Companions tab read "No companions travel with the party yet." TableForge's companion memory therefore remains a vendor claim, unobserved; no page addresses companion death.

None of six reviewed products stated five properties of crypt. None stated open-source code with a fully offline, rules-only mode; all six are hosted browser or app services. None stated a bridge that lets the player's own model, such as Claude Code over MCP, run the game. Hidden Door's Creator MCP authors worlds, and its own engine runs play. None stated that companion memory is checked against engine-recorded state; TableForge's campaign memory is stored game state, and its pages do not say how companions use it. None stated permanent death for companions that no reload undoes; Hidden Door's permadeath setting applies to characters, and its page does not address companions or reloads. None stated generated panels or a layered score; TableForge and Friends & Fables generate art, battle maps, and voice, and Questie gives each companion its own voice. Each absence rests on one to three vendor pages per product, so it records what the vendors state.

In open source, at least eight projects keep dice, hit points, and state in code and let a model narrate or propose actions [16, 17, 29, 30, 18, 19, 31, 32]. kliuchnikovv/dnd is the nearest in form. It runs a Go engine in a terminal that never calls a model, and it still plays offline with its narrator disabled [16]. Mnehmos/rpg-mcp runs as an MCP server in Claude Code under the rule that models propose and never execute [17]. Their GitHub star counts are between 0 and 46 for the repositories whose counts were collected. The open-game-master entry is a fork of Lebbitheplow/open-game-master [19], so its star count is not meaningful. TaleWeave AI is a multiplayer text adventure with AI characters whose model drives character actions with less engine authority [33].

Outside games, four neighbors set expectations. Generative Agents is the research reference for agents with memory, reflection, and planning, set in a simulated town where nobody can die [21]. NVIDIA ACE puts model-driven characters into graphical games, including inZOI, where an on-device small language model drives companion characters [34]. AI Dungeon is a different category, collaborative storytelling with no rules engine, with company-reported 8M+ registered players [35]. Google's A2UI is the architecture analogue for generated panels: a model emits a declarative description from a trusted widget catalog, and the client renders it [36]. The survey by Gallotta et al. maps these roles, player, character, game master, and designer, across the field [37].

TableForge, or any of the open-source projects, could add engine-checked companion memory or permanent companion death, and crypt holds no structural barrier against that. A learning

project does not need one. The party is where the curriculum goes deepest, and that holds whatever other games ship.

Technical

Q18: What are the major technical risks?

Seven. Six are open risks on Must Do features, and each has a spike and a gate before the work that depends on it.

Companion memory fidelity. A companion that misremembers who sold the amulet breaks the promise. Memory is retrieval over the engine's recorded events and the companion's own reflections, and every memory a companion cites is checked against the event log. Spike: the validation run in FAQ 24. Gate: memory-claim accuracy of 95 percent; below 80 percent, retrieval is redesigned (see FAQ 25).

Per-turn latency and cost at three to five companions. Three companions run three agents, plus the narrator, on every turn, and five run five. Batching companion decisions into one call, running companions on a small model, and letting them decide while the player reads are the planned answers. Spike: measure p95 turn latency and cost per turn with three and with five companions on each tier. Latency gate, for p95 with three companions on the hosted tier: under 4 seconds is the target. Between 4 and 8 seconds, companion calls are batched into one request per turn before any presentation tier ships. Above 8 seconds, the solo default drops to two companions, batching is mandatory, and the panels tier stays blocked. Cost gate: the same spike records tokens and cost per turn at three companions on the hosted tier. That number times a monthly turn budget sets the hosted spending ceiling, and the Viability rating moves off MEDIUM when the ceiling is written into the deployment config.

Scenario solvability. A generated dungeon must have a key for every locked door, a way past every trap, prices a party can pay, and cursed items whose curse can be lifted. Spike: generate a batch of dungeons, run the engine's solvability walk on each, and play a sample with automated parties. Gate: every generated dungeon passes the walk before it opens, and a failure falls back to a hand-authored scenario.

Multiplayer transport. The transport for a shared party is open (DES-034), and the daemon's concurrent sessions are the starting point. Spike: two clients in one party over the chosen transport, with turn order enforced by the engine. Gate: no transport work starts until the single-companion validation run passes.

Model quality for scenario generation on the small tier: whether a small or local model can write a scenario and play a companion well enough, or whether only a large model will do. The leaderboard answers this per tier. Gate: a small-tier generated scenario passes the solvability walk on the first try at least 70 percent of the time; below that, small-tier play uses bundled scenarios.

The integration boundary: a companion will try moves the rules forbid, and the engine has to refuse them without the refusal breaking character. Gate: under 5 illegal attempts per 100 companion turns.

Generated presentation: latency and session-long acceptance of a screen that changes every turn are both unanswered, and each presentation tier waits on its own gate (see FAQ 21).

Q19: How do companions run in each mode, and where do their model calls go?

Three modes, three answers. On the hosted service, the daemon runs every companion on the model the operator configures. Every one of those calls records latency, tokens, cost, eval score, and errors in the AWS monitoring. In the local terminal client, the daemon runs companions on the player's configured model or on the small local model. In the rules-only tier, offline or hosted at the spending ceiling, companions follow scripted tactics and make no model calls. No model generates dungeons there either, so the game serves hand-authored scenarios from the bundled set, which is finite. Through the Claude Code bridge, Claude is the interpreter and the narrator, and the companions run inside the same Claude session through the engine's tools. Their model calls are the player's own, so the project's monitoring sees only the engine events those calls produce and never the calls themselves. Evaluation of bridge play uses the engine's event log alone.

Q20: What are the dependencies and their fallbacks?

Six. vox, the Punt Labs text-to-speech tool, gives each companion a voice; without it, the game is text only. ElevenLabs Music generates the layered score; without it, there is no score. AWS is the deployment target for the hosted service, and at hypothesis stage the project needs no second cloud as a fallback. Model providers serve the large and small tiers; the tiered design already falls back from a large model to a small local one to rules-only play. Claude Code is one client among three, so the terminal client and the hosted service work without it. The multiplayer transport is open per DES-034, and the release ships with it. If the transport slips, single-player with companions ships first, and friends join once the transport works.

Q21: How does generated presentation earn its place in the game?

By passing a gate the project measures before the tier ships. Presentation deepens in three tiers, in ascending risk.

Every tier is measured against tempo. TableForge's wait, from Send to text and from Send to voice, is the baseline the predictive cache is measured against; a follow-up timing run of ten turns will put a number on it [7]. Click count per turn is the second tempo metric: the target is one keypress per action and zero clicks to see party state.

Voice comes first. Each companion speaks in its own voice through vox, the text-to-speech tool Punt Labs already ships. TableForge voiced about four long sentences per turn [7]. crypt's spoken narration defaults to at most two sentences per turn, each under 15 words, in the Zork register where a room is two lines and the prompt returns; a "tell me more" keypress requests the long version. Sentences per spoken turn and words per sentence are the Chapter 5 prompt targets, synthesis starts on the first sentence, and the gate is integration.

A layered adaptive score comes second: stems that enter and leave as the scene shifts from corridor to combat to treasure room. Generation uses ElevenLabs Music, with Audio Reference steering each new stem toward the tempo, instrumentation, and mood of the ones already playing [38]. The vendor does not claim key matching, so key is part of what the gate measures. Its gate is coherence: stems generated across a full session still sound like one piece of music to a listening judge.

Generated panels come third: the screen around the text redraws to fit the scene, with a map, a party roster, or a ledger of loot. The model fills a schema from a fixed widget vocabulary, the A2UI pattern, and the client renders it [36]. In pairwise tests, people prefer generated pages to markdown answers [39], and they prefer generated task-specific interfaces in over 70% of cases [40]. Both studies measure one-shot pages for information tasks, so their results do not transfer

directly to a game screen that persists across a session. Two risks remain open. Latency is the documented failure: Gemini's dynamic view takes a minute or more to generate [41]. Session-long acceptance of a screen that changes every turn has no measurement anywhere, and Nielsen Norman Group names lost consistency as the cost of generated UI [42]. The design answer to consistency is a fixed frame: the layout holds still, and only the contents of its panels regenerate. The answer to latency is a predictive cache. Turn-based play gives the game dead time while the player reads, and the engine already enumerates the legal next states, so the set of screens to prepare per turn is small. The cost of a prepared panel is measured in the latency spike for this tier, because no published figure covers schema-filled panels. The gate for panels is a cache hit rate and a p95 latency under a budget fixed before the tier is built.

Generated images in terminals that support them are a fourth, optional step. Terminal image protocols are fragmented across Kitty, iTerm2, and Sixel [43, 44, 45], so the text and half-block display stays the baseline, and the gate is reliable protocol detection. For latency, Unbounded reached about one second per generated scene image with distillation [46], which is the reference point for this tier.

Q22: What happens if generated presentation fails its gates?

If two or more presentation tiers fail their gates, the project turns to an agent-authored world. The part of crypt that sets it apart then has to come from the dungeon growing, because presentation cannot supply it. AI wizards, in the LPMud sense of builders with the power to add to the world [3], dig new rooms, monsters, and puzzles overnight. The engine validates every build for solvability and balance before it opens, and companion parties play-test it before a human sees it. SpacetimeMUD already has an agent building rooms through the player interface at demo scale [47]. The fallback exercises the same curriculum from another direction: planning agents, exact evaluation of their output, and a feedback flywheel from player ratings. Its risk is quality, as critiques of the agent-populated network Moltbook show [48]. It is the last Should Do item, and it starts only on that trigger (see Feature 19).

Q23: What is the state of the build, and what remains?

Working today: the deterministic engine, the daemon that serves it with concurrent sessions, the thin client with a terminal UI, the Claude Code bridge, and the tiered model integration. vox, the voice tool companions use, ships separately. The order of the remaining work is content first, presentation second. The validation run comes first (see FAQ 24). Companions with memory and permanent death follow, then generated scenarios with engine-proved solvability, then the hosted service on AWS with monitoring, invite-only behind a monthly ceiling, and the multiplayer transport, so friends join on launch day. The fine-tuning experiment runs once the harvested corpus exists. Presentation follows in gate order: voice, music, panels, images. If scope compresses, presentation tiers are cut from the top down, and the party and production stay.

Q24: What is the next step to validate the plan?

One companion, one scripted dungeon, repeated sessions, before any multiplayer transport work. The run scores three things. Rule-following: the engine accepts zero illegal moves, and the run counts how often the companion attempts one. Memory: every claim the companion makes about an earlier event is checked against the engine's event log. Latency: p95 turn time on each model tier with one companion; the three-companion latency and cost figures come

from the spike in FAQ 18. The thresholds in FAQ 25 decide whether companions go forward as designed, need re-prompting, or need a retrieval redesign. This run is the first mission.

Scope & Success

Q25: What does success look like for a learning project?

Two things. A documented understanding of where the model helps, where it falls short, how to measure its performance, and what it takes to improve it. And a public, playable game running in production on AWS, recording per-call latency, tokens, cost, eval score, and error rate, in which a nostalgic player enjoys a party. The game is where those lessons get tested. A working evaluation suite, a defensible answer to the code-versus-model boundary, companions that keep their memories and their rules, and hands-on experience with serving, monitoring, tuning, and fine-tuning are the deliverables.

Five thresholds turn that into decisions.

Companion illegal moves. The engine accepts none, so the measure is attempts per 100 companion turns. Target: under 5. Above that, the companion prompts are reworked before companions go further.

Memory-claim accuracy against engine-recorded events. Target: 95 percent. Below 80 percent, companion memory retrieval is redesigned.

p95 turn latency with three companions on the hosted tier. Target: under 4 seconds. Between 4 and 8 seconds, companion calls are batched into one request per turn before any presentation tier ships. Above 8 seconds, the solo default drops to two companions, batching is mandatory, and the panels tier stays blocked.

Cost per turn. The latency spike (FAQ 18) records tokens and cost per turn at three companions on the hosted tier. That number times a monthly turn budget sets the hosted ceiling, and the Viability rating moves off MEDIUM when the ceiling is written into the deployment config.

Real players for the monitoring and feedback chapters. Minimum: 10 returning players over a month. A returning player is one saved party that plays on two or more distinct days in a month; the save identity, not the invite link, is the unit. Below that, those chapters run on persona play-testers, and their results are labeled as persona results.

Risk Assessment

Risk	Rating	Assessment
Value	LOW	The project is four parts learning to one part game, so its value is the engineering knowledge gained, and the work produces that knowledge whether or not players arrive. Player demand has a floor: the monitoring and feedback chapters need 10 returning players over a month. Below that, they run on persona play-testers and report persona results (see FAQ 16).
Usability	MEDIUM	Four open usability risks, each with a measurement. Parser frustration, the known failure of the genre: the interpreter tries the rules parser first and calls a model only for input the rules cannot parse. Measure: the share of player inputs the rules interpreter resolves without a model, target over 80 percent on the bundled scenarios. Companion chatter: one companion speaks unprompted per turn, and the persona play-testers rate readability. Waiting on agent calls: companions decide while the player reads, and the p95 latency gate in FAQ 18 sets what happens above 4 and 8 seconds. Prompt injection by other humans in the party: the engine bounds what injection can do to outcomes, so an injected instruction cannot reposition a companion, change a roll, or alter inventory. What it can change is what a companion or the narrator says. On the hosted service, generated text passes an output check before it reaches other players, one of the Chapter 10 guardrails exercises [20].
Feasibility	MEDIUM	The engine, daemon, client, concurrent sessions, and model integration already work. Six Must Do risks are open, each with a spike and a gate: companion memory fidelity; per-turn latency and cost at three to five companions; scenario solvability for doors, keys, shops, traps, and cursed items; the multiplayer transport, open per DES-034, which the release ships with; model quality for small-tier scenario generation; and the integration boundary (see FAQ 18). Presentation tiers each ship only through their own gate (see FAQ 21).
Viability	MEDIUM	No revenue model to satisfy, but the hosted service spends money on every model call. It launches invite-only with a monthly spending ceiling. At the ceiling, the service drops to the small-model tier, then to rules-only play, and new invites pause. The three-companion latency spike records tokens and cost per turn on the hosted tier; that number times a monthly turn budget sets the ceiling. The rating moves off MEDIUM when the ceiling is written into the deployment config (see FAQ 8 and FAQ 25).

Feature Appendix

Must Do

- **F1.** **Party of six**: any mix of human players and companions fills the six seats; a solo party starts with three companions and can seat up to five, and a friend can take a companion's seat between descents
- **F2.** **Companions**: model-played characters with a personality and memory across sessions, acting only through the engine tools a human player uses; memory is retrieval over engine-recorded events and the companion's own reflections; death cannot be undone by reloading a save, and resurrection, where the dungeon offers it, follows the same rules and costs as for humans
- **F3.** **Multiplayer transport**: friends in different places share one party; the transport is open per DES-034, starting from the daemon's concurrent sessions, and ships in the release (see FAQ 20)
- **F4.** **Deterministic engine**: rules, combat, inventory, movement, death, save and load, run in code and never in a model
- **F5.** **Daemon and tiered model integration**: large model, small local model, and a no-model fallback, so the game is always playable
- **F6.** **Free-text interpretation**: the model turns plain-language input into engine actions
- **F7.** **Generated narration**: the model describes rooms, events, and characters on demand
- **F8.** **Claude Code bridge**: Claude plays the dungeon master through the stdio bridge into the engine, and companions run inside the same Claude session through the engine's tools (see FAQ 19)
- **F9.** **Generated scenarios**: the model writes a new dungeon for each descent, with shops, traps, and cursed items, and the engine proves it solvable before it opens
- **F10.** **Evaluation suite and private leaderboard**: scored, repeatable measurement of interpreter accuracy against a hand-labeled intent set, companion rule-following and memory claims against engine-recorded events, judge-scored decision quality and personality consistency, and game balance
- **F11.** **Fine-tuning experiment**: train one small model on the harvested corpus and compare it to prompting on the leaderboard, whatever the outcome; this is Huyen's fine-tuning chapter in practice
- **F12.** **AWS production**: the game runs in production on AWS, recording per-call latency, tokens, cost, eval score, and error rate for every model call, the system Huyen's inference-optimization and monitoring chapters practice on
- **F13.** **Hosted service and public availability**: a stranger can join a hosted party with an invite link, with no account at launch, or install and play without a developer checkout; the hosted service launches invite-only under a monthly spending ceiling, and at the ceiling it drops to the small-model tier, then to rules-only play, and new invites pause

Should Do

Presentation tiers ship in this order, lowest feasibility risk first, each when its gate passes (see FAQ 21). The agent-authored world comes last, on its trigger.

- **F14.** **Per-companion voice**: each companion speaks in its own voice through vox; gate: integration only
- **F15.** **Layered adaptive music**: stems that follow the scene, generated with ElevenLabs Music under Audio Reference; gate: stems stay coherent across a full session
- **F16.** **Fixed-frame generated panels**: the layout holds still and panel contents regenerate from a fixed widget vocabulary, prepared ahead by a predictive cache; gate: cache hit rate and p95 latency under a stated budget
- **F17.** **Generated images**: scene images in terminals that support an image protocol, with text and half-block display as the baseline; gate: reliable protocol detection across fragmented terminal support
- **F18.** **Fine-tuned small model shipped**: a model tuned for the game's narrow narration, parsing, and companion tasks ships only if the fine-tuning experiment shows it beats prompting and a stock small model on the leaderboard (see Feature 11)
- **F19.** **Agent-authored persistent world**: AI wizards that dig new rooms overnight, validated by the engine and play-tested by companion parties; pursued if two or more presentation tiers fail their gates, because the dungeon's growth then has to set crypt apart (see FAQ 22)

Won't Do

- **F20.** **Model-decided game rules**: no model, narrator or companion, ever resolves combat, damage, or death; that is the one boundary the design refuses to cross
- **F21.** **Graphical 3D client**: the game is text-first by identity; a rendered 3D world is a different product
- **F22.** **Real-time action**: crypt is turn-based like its ancestors; twitch gameplay is out of scope
- **F23.** **Monetization**: a build-to-learn project has no store, subscription, or paid tier

References

[1] Jimmy Maher. *ZIL and the Z-Machine*. The Digital Antiquarian. Z-machine design, paging, 32K target, content cuts (191 to 110 rooms, 911 to 617 words). 2012. URL: https://www.filfre.net/2012/01/zil-and-the-z-machine/ (visited on 11/09/2026).

[2] Wikipedia. *Wizardry: Proving Grounds of the Mad Overlord*. Tertiary. Authors, 1978 start, party of six, classes, ten levels, sales figures, Japan reception. 2026. URL: https://en.wikipedia.org/wiki/Wizardry:_Proving_Grounds_of_the_Mad_Overlord (visited on 11/09/2026).

[3] Wikipedia. *LPMud*. Tertiary. Pensjo 1989, LPC, driver and mudlib, wizard coding, MudOS and DGD. 2026. URL: https://en.wikipedia.org/wiki/LPMud (visited on 11/09/2026).

[4] Wikipedia. *LambdaMOO*. Tertiary. Curtis, Xerox PARC, 1990; about 10,000 users near 1994. 2026. URL: https://en.wikipedia.org/wiki/LambdaMOO (visited on 11/09/2026).

[5] Pengfei Yu et al. *RPGBench: Evaluating Large Language Models as Role-Playing Game Engines*. Rule-based plus LLM-judge evaluation; models write engaging stories but keep mechanics inconsistent (v). Submitted to ICML 2025; acceptance unconfirmed. 2025. arXiv: 2502.00595. URL: https://arxiv.org/abs/2502.00595.

[6] Latitude. *Why does the AI forget or mix things up?* AI Dungeon help page. Context limits trim older story text. 2025. URL: https://help.aidungeon.com/faq/why-does-the-ai-forget-or-mix-things-up (visited on 11/09/2026).

[7] Jim Freeman. *TableForge hands-on session*. research/research-2026-11-09-tableforge-handson.md. Primary. One free-tier campaign played in Safari on macOS, 2026-11-09, seven screenshots. Operator observations on tempo, voice-on-click, portrait animation, and absent companions. Saved as research/research-2026-11-09-tableforge-handson.md. 2026.

[8] Latitude. *Do you support Multiplayer?* Primary vendor help page. No set player limit; host's settings apply to all players. 2026. URL: https://help.aidungeon.com/faq/do-you-support-multiplayer (visited on 11/09/2026).

[9] TableForge. *TableForge: AI Dungeon Master for D&D 5e*. Primary vendor page. Separate rules engine, companions per plan, pricing $0 to $39.99 per month, party size up to 6. 2026. URL: https://tableforge.gg/ (visited on 11/09/2026).

[10] TableForge. *TableForge FAQ*. Primary vendor page. Rules engine governs dice, HP, spell slots, conditions, combat. No mention of permadeath, offline, open source, or MCP. 2026. URL: https://tableforge.gg/faq (visited on 11/09/2026).

[11] TableForge. *Best Multiplayer AI Dungeon Master for a Group Without a DM*. Vendor-authored comparison of TableForge, Friends & Fables, The Endlessness, LoreKeeper, AI Dungeon, Roll20, Foundry. Published Aug 14, 2026, updated Oct 20, 2026. Tertiary and self-interested for competitor claims. 2026. URL: https://tableforge.gg/blog/multiplayer-ai-dungeon-master-comparison (visited on 11/09/2026).

[12] Friends & Fables. *Friends & Fables: AI TTRPG and World Building Game*. Primary vendor page. AI GM Franz narrates and adjudicates rules; up to 6 players; SRD 5.1. 2026. URL: https://fables.gg (visited on 11/09/2026).

[13] Hidden Door. *Hidden Door*. Primary vendor page. Story engine with canon check, Death setting None/Normal/Permadeath, Creator MCP. 2026. URL: https://hiddendoor.co (visited on 11/09/2026).

[14] Auferet. *Multiplayer AI RPG*. Primary vendor page. Dice roller and stat tracking; AI decides outcomes; four group modes; free plus paid from $10 per month. 2026. URL: https://auferet.com/multiplayer-ai-rpg (visited on 11/09/2026).

[15] Questie.ai. *Multiplayer AI Chatbot and Party Mode*. Primary vendor page. One human plus up to four voiced AI companions; Legend plan only; d20 roll with companion reactions. 2026. URL: https://www.questie.ai/multiplayer-ai-party-mode (visited on 11/09/2026).

[16] kliuchnikovv. *dnd: a console detective RPG with a deterministic Go rules engine and an LLM narrator*. MIT, Go, 0 stars, 332 commits. Core never calls a model; fallback chain; offline with narrator disabled. Year is an estimate. 2026. URL: https://github.com/kliuchnikovv/dnd (visited on 11/09/2026).

[17] Mnehmos. *rpg-mcp: rules-enforced RPG backend for LLM game masters*. TypeScript, 46 stars, 128 commits. License conflicts between sidebar (MIT) and README (ISC). "LLMs propose, never execute." Year is an estimate. 2026. URL: https://github.com/Mnehmos/rpg-mcp (visited on 11/09/2026).

[18] slowbutfast. *open-dungeon: open source AI Dungeon with an engine-validated LLM narrator*. MIT, Node and Python, 0 stars, 178 commits. Web app plus MCP server; MOCK_LLM mode needs no model. Year is an estimate. 2026. URL: https://github.com/slowbutfast/open-dungeon (visited on 11/09/2026).

[19] AlecVlad. *open-game-master (fork of Lebbitheplow/open-game-master; formerly Open Dungeon Master)*. MIT, TypeScript, 3,032 commits. Server-side engines own 5e rules; local-first. Star count on this fork not meaningful. Year is an estimate. 2026. URL: https://github.com/AlecVlad/open-game-master (visited on 11/09/2026).

[20] Chip Huyen. *AI Engineering: Building Applications with Foundation Models*. Curriculum for the project. Evaluation-driven development, AI-as-a-judge and its drift, comparative over absolute evaluation. O'Reilly Media, 2025. ISBN: 9781098166304.

[21] Joon Sung Park et al. "Generative Agents: Interactive Simulacra of Human Behavior". In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. Peer-reviewed. 25 LLM agents with memory, reflection and planning in a Sims-like town. 2023. DOI: 10.1145/3586183.3606763. arXiv: 2304.03442. URL: https://arxiv.org/abs/2304.03442.

[22] Andrew Zhu et al. "FIREBALL: A Dataset of Dungeons and Dragons Actual-Play with Structured Game State Information". In: *Proceedings of ACL 2023*. Peer-reviewed. About 25,000 D&D sessions with Avrae game state; state improves narration. 2023, pp. 4171–4193. arXiv: 2305.01528. URL: https://arxiv.org/abs/2305.01528.

[23] Jaewoo Song, Andrew Zhu, and Chris Callison-Burch. "You Have Thirteen Hours in Which to Solve the Labyrinth: Enhancing AI Game Masters with Function Calling". In: *Wordplay Workshop at ACL 2024*. Function calling improves narrative quality and state consistency; human evaluation plus unit tests (v). 2024. arXiv: 2409.06949. URL: https://arxiv.org/abs/2409.06949.

[24] Ivan Pelivanovic. *Dungeons & Dragons (D&D) Meets AI: A Smarter Game Engine*. Industry blog, 29 Sep 2025. Infobip hybrid engine; pure-LLM accuracy fell below 65 percent with context length. Vendor self-report. 2025. URL: https://shiftmag.dev/dungeons-dragons-dnd-ai-game-engine-6240/ (visited on 11/09/2026).

[25] Gaëtan Berlaimont et al. *CriticaLLM: Using LLMs as Autonomous Game Master for Tabletop Role-Playing Games*. Preprint. LLM Game Master plus Judge LLM that validates input; rules enforced by an LLM, not code. 2026. arXiv: 2610.21525. URL: https://arxiv.org/abs/2610.21525.

[26] Ian Bicking. *Intra: design notes on an LLM-driven text adventure*. Practitioner blog, 1 Jul 2025. Failure modes of LLM-run text games and partial state separation. 2025. URL: https://ianbicking.org/blog/2025/07/intra-llm-text-adventure (visited on 11/09/2026).

[27] Jules Clerc et al. "Creating Hybrid Interactive Narrative Systems with LLM: From Formal Description to Evaluation". In: *38th IEEE International Conference on Tools with Artificial Intelligence (ICTAI 2026)*. Accepted venue per arXiv listing. Hybrid experience managers: LLM generation with symbolic world model. 2026. arXiv: 2611.02807. URL: https://arxiv.org/abs/2611.02807.

[28] *EvoGenUI-Bench: Evaluating LLMs as Multi-Turn Generative UI Assistants*. Multi-turn consistency of UI, state and claims. Authors not verified; see arXiv 2608.29387. 2026. arXiv: 2608.29387. URL: https://arxiv.org/abs/2608.29387.

[29] thegreystone. *rpg-mcp: persistent rules-aware RPG engine as an MCP server*. Java (Quarkus), BSD-3-Clause, 5 stars, 90 commits. Engine owns dice and state; audited GM override. Year is an estimate. 2026. URL: https://github.com/thegreystone/rpg-mcp (visited on 11/09/2026).

[30] shadowgate15. *the-dungeon-master: an AI dungeon master with deterministic rules and state*. TypeScript, SQLite, 0 stars, 64 commits. Application license not stated; SRD 5.1 content CC-BY-4.0. Year is an estimate. 2026. URL: https://github.com/shadowgate15/the-dungeon-master (visited on 11/09/2026).

[31] datashaman. *ai-ttrpg: An AI-assisted tabletop RPG engine with deterministic rules, event-sourced state, and LLM-powered interpretation and narration*. Open-source repo. "The LLM proposes; the engine disposes." Year is an estimate; not fetched directly. 2026. URL: https://github.com/datashaman/ai-ttrpg (visited on 11/09/2026).

[32] undergroundrap. *SINGLE-PLAYER-AI-MUD*. Open-source repo, AGPL-3.0, 2 stars. Deterministic Python rules, local LLM for flavor. Year is an estimate. 2026. URL: https://github.com/undergroundrap/SINGLE-PLAYER-AI-MUD (visited on 11/09/2026).

[33] ssube. *TaleWeave AI*. Open-source repo, MIT, 14 stars. Multiplayer text adventure and MUD engine with AI NPCs. Year is an estimate. 2024. URL: https://github.com/ssube/taleweave-ai (visited on 11/09/2026).

[34] NVIDIA. *NVIDIA ACE Autonomous Game Characters Launch in NARAKA: BLADEPOINT MOBILE and inZOI This Month*. Vendor announcement, 13 March 2025. Smart Zoi companions in inZOI driven by an on-device half-billion-parameter ACE small language model (Mistral-Nemo-Minitron Instruct); inZOI early access 28 March 2025; RTX GPUs required. 2025. URL: https://www.nvidia.com/en-us/geforce/news/gfecnt/20253/nvidia-ace-naraka-bladepoint-inzoi-launch-this-month/ (visited on 11/09/2026).

[35] Latitude. *AI Dungeon and Latitude company site and blog*. Company-reported: 8M+ registered players, 68M+ adventures; DAU up over 70 percent in six months (June 2025). Not audited. 2025. URL: https://latitude.io/ (visited on 11/09/2026).

[36] Google A2UI Team. *Introducing A2UI: An Open Project for Agent-Driven Interfaces*. Published 2025-12-15. Declarative JSON UI from a trusted widget catalog; incremental agent updates across a conversation; adopters CopilotKit, Opal, Flutter GenUI SDK (v). 2025. URL: https://developers.googleblog.com/introducing-a2ui-an-open-project-for-agent-driven-interfaces/ (visited on 11/09/2026).

[37] Roberto Gallotta et al. "Large Language Models and Games: A Survey and Roadmap". In: *IEEE Transactions on Games* (2024). Survey of LLM roles in games including game master and PCG. DOI: 10.1109/TG.2024.3461510. arXiv: 2402.18659.

[38] ElevenLabs. *Music*. Vendor documentation (fetched). Music v2 and v2.5; Audio Reference guides a generation from an uploaded clip of up to about 30 seconds and influences sound, production style, instrumentation, tempo, and mood; the page does not claim key matching and says it does not copy or remix the reference. 2026. URL: https://elevenlabs.io/docs/eleven-creative/products/music (visited on 11/09/2026).

[39] Yaniv Leviathan et al. *Generative UI: LLMs are Effective UI Generators*. Google Research. Gemini 3 Pro generates interactive pages per prompt; preferred over markdown, comparable to human experts in 50 percent of cases; PAGEN dataset. Preprint (v). 2026. arXiv: 2604.09577. URL: https://arxiv.org/abs/2604.09577.

[40] Jiaqi Chen et al. *Generative Interfaces for Language Models*. Stanford. LLM-generated task-specific UIs preferred by humans in over 70 percent of cases; accepted to ACL 2026 Findings (per search). 2025. arXiv: 2508.19227. URL: https://arxiv.org/abs/2508.19227.

[41] Google Research. *Generative UI: A Rich, Custom, Visual Interactive User Experience for Any Prompt*. Dynamic view and visual layout in Gemini app and AI Mode; latency of a minute or more; evaluation excluded generation speed (v). 2025. URL: https://research.google/blog/generative-ui-a-rich-custom-visual-interactive-user-experience-for-any-prompt/ (visited on 11/09/2026).

[42] Kate Moran and Sarah Gibbons. *Generative UI and Outcome-Oriented Design*. Nielsen Norman Group. Names consistency and predictability as the cost of per-use generated UI. Opinion, no data (search summary). 2024. URL: https://www.nngroup.com/articles/generative-ui/ (visited on 11/09/2026).

[43] Kovid Goyal. *Terminal graphics protocol*. Primary spec: transmission modes, placements, Unicode placeholders, list of implementing terminals. 2026. URL: https://sw.kovidgoyal.net/kitty/graphics-protocol/ (visited on 11/09/2026).

[44] iTerm2. *Inline Images Protocol*. Primary spec: OSC 1337 File, multipart, size units, 1 MiB and tmux limits. 2026. URL: https://iterm2.com/documentation-images.html (visited on 11/09/2026).

[45] Are We Sixel Yet. *Are We Sixel Yet?* Community-maintained Sixel support table by terminal; secondary source. 2026. URL: https://www.arewesixelyet.com/ (visited on 11/09/2026).

[46] Jialu Li et al. "Unbounded: A Generative Infinite Game of Character Life Simulation". In: *International Conference on Learning Representations (ICLR)*. LLM game engine plus diffusion images; about 1 second per scene versus about 5 seconds naive. 2025. arXiv: 2410.18975.

[47] Clockwork Labs. *SpacetimeMUD*. Open-source repo, 14 stars. AI agent builds rooms and NPCs via OpenAI API. Year is an estimate. 2024. URL: https://github.com/clockworklabs/spacetime-mud (visited on 11/09/2026).

[48] Wikipedia. *Moltbook*. Launch 2026-01-28, Schlicht, OpenClaw link, scale, authenticity disputes, Meta acquisition. Tertiary. 2026. URL: https://en.wikipedia.org/wiki/Moltbook (visited on 11/09/2026).